

Project 1: Dolly Zoom 实验报告

姓名: 李祥熙 学号: 2234412799 班级: 计算机 2301

目录

1 实验目的	2
2 实验过程	2
2.1 实验环境	2
2.2 实验场景描述	2
2.3 实现步骤	2
3 实验内容	3
3.1 小孔成像模型	3
3.2 任务一: <code>compute_focal_length</code> ——给定相机位移, 计算新焦距	3
3.2.1 问题描述	3
3.2.2 数学推导	3
3.2.3 代码实现	4
3.2.4 测试验证	4
3.3 任务二: <code>compute_f_pos</code> ——给定两物体像高之比, 求相机位置与焦距	4
3.3.1 问题描述	4
3.3.2 数学推导	5
3.3.3 代码实现	6
3.3.4 测试验证	6
4 实验结果	7
4.1 测试结果汇总	7
4.2 结果分析	8
5 总结	8

1 实验目的

本实验的目标如下：

- **掌握小孔成像（针孔相机）数学模型：**理解三维空间中的点如何通过透视投影变换映射到二维图像平面，以及焦距、物距与像高之间的关系。
- **理解 Dolly Zoom 效果的原理：**Dolly Zoom（推拉镜头效果）是电影工业中常用的视觉技巧，其核心在于同步调整摄像机的位置和焦距，使感兴趣物体在画面中保持固定大小，同时令背景或前景物体产生显著的缩放变化，营造出眩晕感。
- **编程实现两个核心函数：**通过推导并实现 `compute_focal_length` 和 `compute_f_pos` 两个函数，将数学模型落地为可执行的代码，加深对透视投影变换的直观认识。

2 实验过程

2.1 实验环境

- 操作系统：Linux（Arch Linux）
- 编程语言：Python 3
- 主要工具：Jupyter Notebook、NumPy

2.2 实验场景描述

本实验使用一个合成三维场景（场景俯视图参见实验说明文档图 1），场景中包含两个立方体（物体 A 和物体 C）和一个锥体（物体 B）。摄像机沿 Z 轴方向运动，场景中的物体保持静止。实验目标是：在摄像机前后移动的过程中，通过同步调整焦距，使感兴趣物体在图像中的投影高度保持恒定，同时控制其他物体的相对大小变化。

2.3 实现步骤

1. 根据小孔成像模型推导像高与焦距、物距之间的关系式。
2. 由“像高不变”约束推导出 `compute_focal_length` 函数的解析公式。
3. 在两个约束条件（像高不变 + 两物体像高之比等于给定值）下，联立方程推导出 `compute_f_pos` 函数的解析公式。
4. 用测试用例验证两个函数的输出是否符合预期。

3 实验内容

3.1 小孔成像模型

根据小孔成像（针孔相机）模型，三维空间中的一个点 (X, Y, Z) 投影到图像平面后的坐标为：

$$u = f \frac{X}{Z} + p_x, \quad v = f \frac{Y}{Z} + p_y \quad (1)$$

其中， f 是以像素为单位的焦距（在 x 和 y 方向相同）， (p_x, p_y) 是主点坐标。

对于一个在物理世界中高度为 H 、距摄像机光心距离为 Z 的物体，其投影到图像平面的像高 h （以像素计）满足：

$$h = f \cdot \frac{H}{Z} \quad (2)$$

这一关系是本实验所有推导的出发点：像高 h 与焦距 f 成正比，与物距 Z 成反比。

3.2 任务一：compute_focal_length —— 给定相机位移，计算新焦距

3.2.1 问题描述

给定感兴趣物体的参考距离 d_{ref} 以及摄像机的初始焦距 f_{ref} ，当摄像机沿 Z 轴移动了位移 pos （向前为正，向后为负）之后，物体与摄像机光心的新距离变为 $d_{\text{ref}} - pos$ 。要求计算新的焦距 f ，使得感兴趣物体在图像中的像高保持不变。

3.2.2 数学推导

初始状态下，物体的像高为：

$$h = f_{\text{ref}} \cdot \frac{H}{d_{\text{ref}}} \quad (3)$$

摄像机移动 pos 之后，若保持像高不变，则需要新焦距 f 满足：

$$f_{\text{ref}} \cdot \frac{H}{d_{\text{ref}}} = f \cdot \frac{H}{d_{\text{ref}} - pos} \quad (4)$$

物体高度 H 可约去，整理得：

$$\boxed{f = f_{\text{ref}} \cdot \frac{d_{\text{ref}} - pos}{d_{\text{ref}}}} \quad (5)$$

该公式的直觉含义十分清晰：若摄像机向物体靠近（ $pos > 0$ ），物距缩短，为保持像高，需要减小焦距；若摄像机远离物体（ $pos < 0$ ），物距增大，则需要增大焦距。

3.2.3 代码实现

```
def compute_focal_length(d_ref, f_ref, pos):
    # 保持像高不变:  $f_{\text{ref}} * H / d_{\text{ref}} = f * H / (d_{\text{ref}} - pos)$ 
    # 约去 H, 解出 f:
    f = f_ref * (d_ref - pos) / d_ref
    return f
```

函数接受物体参考距离 d_{ref} 、初始焦距 f_{ref} 、以及摄像机沿 Z 轴的位移数组 pos (支持标量或 NumPy 数组), 返回对应的新焦距 f 。由于公式仅包含基本算术运算, 当 pos 为数组时, NumPy 会自动进行广播, 一次性计算出每个位置对应的焦距, 这对于生成 Dolly Zoom 动画序列 (逐帧计算焦距) 非常高效。

3.2.4 测试验证

使用以下测试参数进行验证:

参数	含义	值
d_{ref}	感兴趣物体初始距离	4
f_{ref}	初始焦距 (像素)	400
pos	摄像机沿 Z 轴的位移	-5 (向后移动)

代入公式 (5):

$$f = 400 \times \frac{4 - (-5)}{4} = 400 \times \frac{9}{4} = 900 \quad (6)$$

验证: 初始像高 $h = 400 \times H/4 = 100H$; 移动后像高 $h' = 900 \times H/9 = 100H$, 两者相等, 说明实现正确。

3.3 任务二: compute_f_pos —— 给定两物体像高之比, 求相机位置与焦距

3.3.1 问题描述

场景中有两个物体:

- 物体 1: 初始距离 $d_{1,\text{ref}}$, 物理高度 H_1 ;
- 物体 2: 初始距离 $d_{2,\text{ref}}$, 物理高度 H_2 。

初始焦距为 f_{ref} 。需要找到一个摄像机位移 pos 和新焦距 f , 满足:

1. 物体 1 的像高在图像中保持不变;
2. 物体 1 与物体 2 在图像中的像高之比等于给定值 $\text{ratio} = h_1/h_2$ 。

3.3.2 数学推导

约束一：物体 1 的像高不变 与任务一完全相同，可以直接得到新焦距与位移的关系：

$$f = f_{\text{ref}} \cdot \frac{d_{1,\text{ref}} - pos}{d_{1,\text{ref}}} \quad (7)$$

约束二：两物体像高之比等于 ratio 摄像机移动 pos 之后，两个物体分别距光心：

$$Z'_1 = d_{1,\text{ref}} - pos \quad (8)$$

$$Z'_2 = d_{2,\text{ref}} - pos \quad (9)$$

对应的新像高分别为：

$$h_1 = f \cdot \frac{H_1}{Z'_1} = f \cdot \frac{H_1}{d_{1,\text{ref}} - pos} \quad (10)$$

$$h_2 = f \cdot \frac{H_2}{Z'_2} = f \cdot \frac{H_2}{d_{2,\text{ref}} - pos} \quad (11)$$

两式相除时，新焦距 f 恰好被约去，得到一个只含 pos 的方程：

$$\frac{h_1}{h_2} = \frac{H_1 / (d_{1,\text{ref}} - pos)}{H_2 / (d_{2,\text{ref}} - pos)} = \frac{H_1 \cdot (d_{2,\text{ref}} - pos)}{H_2 \cdot (d_{1,\text{ref}} - pos)} = \text{ratio} \quad (12)$$

对方程 (12) 进行展开和整理：

$$H_1(d_{2,\text{ref}} - pos) = \text{ratio} \cdot H_2 \cdot (d_{1,\text{ref}} - pos) \quad (13)$$

$$H_1 d_{2,\text{ref}} - H_1 \cdot pos = \text{ratio} \cdot H_2 \cdot d_{1,\text{ref}} - \text{ratio} \cdot H_2 \cdot pos \quad (14)$$

$$pos \cdot (\text{ratio} \cdot H_2 - H_1) = \text{ratio} \cdot H_2 \cdot d_{1,\text{ref}} - H_1 d_{2,\text{ref}} \quad (15)$$

最终解得：

$$\boxed{pos = \frac{H_1 \cdot d_{2,\text{ref}} - \text{ratio} \cdot H_2 \cdot d_{1,\text{ref}}}{H_1 - \text{ratio} \cdot H_2}} \quad (16)$$

求得 pos 之后，代入公式 (7) 即可得到新焦距 f 。

关键观察： 公式 (12) 中焦距 f 被约去，这说明两物体像高之比仅取决于摄像机的位置，而与焦距无关。因此，可以先由比例约束单独解出 pos ，再由“像高不变”约束确定 f ，两个未知量被完全解耦，求解过程简洁。

3.3.3 代码实现

```
def compute_f_pos(d1_ref, d2_ref, H1, H2, ratio, f_ref):
    # 约束 2: h1/h2 = ratio, f 被约去
    # H1*(d2_ref-pos) = ratio*H2*(d1_ref-pos)
    # 整理得 pos:
    pos = (H1 * d2_ref - ratio * H2 * d1_ref) / (H1 - ratio * H2)

    # 约束 1: 物体 1 像高不变
    # f_ref * H1 / d1_ref = f * H1 / (d1_ref - pos)
    f = f_ref * (d1_ref - pos) / d1_ref

    return f, pos
```

函数先利用公式 (16) 计算出摄像机应移动到的位置 pos ，再利用公式 (5) 计算对应的新焦距 f ，返回 (f, pos) 。

3.3.4 测试验证

使用以下测试参数进行验证：

参数	含义	值
d1_ref	物体 1 初始距离	4
d2_ref	物体 2 初始距离	20
H1	物体 1 物理高度	4
H2	物体 2 物理高度	6
ratio	目标像高之比 h_1/h_2	2
f_ref	初始焦距（像素）	400

计算 pos ： 代入公式 (16)：

$$pos = \frac{4 \times 20 - 2 \times 6 \times 4}{4 - 2 \times 6} = \frac{80 - 48}{4 - 12} = \frac{32}{-8} = -4 \quad (17)$$

摄像机需要向后（远离物体方向）移动 4 个单位。

计算 f ： 代入公式 (5)：

$$f = 400 \times \frac{4 - (-4)}{4} = 400 \times 2 = 800 \quad (18)$$

验证约束一（物体 1 像高不变）：

$$h_{1,\text{初始}} = f_{\text{ref}} \cdot \frac{H_1}{d_{1,\text{ref}}} = 400 \times \frac{4}{4} = 400 \text{ px} \quad (19)$$

$$h_{1,\text{新}} = f \cdot \frac{H_1}{d_{1,\text{ref}} - pos} = 800 \times \frac{4}{4 - (-4)} = 800 \times \frac{4}{8} = 400 \text{ px} \quad (20)$$

两者相等，约束一满足。✓

验证约束二（像高之比等于 ratio）：

$$h_{2,\text{新}} = f \cdot \frac{H_2}{d_{2,\text{ref}} - pos} = 800 \times \frac{6}{20 - (-4)} = 800 \times \frac{6}{24} = 200 \text{ px} \quad (21)$$

$$\frac{h_{1,\text{新}}}{h_{2,\text{新}}} = \frac{400}{200} = 2 = \text{ratio} \quad (22)$$

约束二同样满足。✓

4 实验结果

4.1 测试结果汇总

表 1: compute_focal_length 测试结果

参数	输入值	计算结果
d_{ref} （物体初始距离）	4	—
f_{ref} （初始焦距）	400	—
pos （摄像机位移）	-5	—
f （新焦距）	—	900

表 2: compute_f_pos 测试结果

参数	输入值	计算结果
$d_{1,\text{ref}}$ （物体 1 初始距离）	4	—
$d_{2,\text{ref}}$ （物体 2 初始距离）	20	—
H_1 （物体 1 物理高度）	4	—
H_2 （物体 2 物理高度）	6	—
ratio（目标比例 h_1/h_2 ）	2	—
f_{ref} （初始焦距）	400	—
pos （摄像机最终位移）	—	-4
f （新焦距）	—	800

4.2 结果分析

- **摄像机向后移动 ($pos < 0$) 时, 焦距增大:** 在 `compute_focal_length` 测试中, 摄像机向后移动了 5 个单位, 焦距从 400 增大到 900, 增幅比例恰好等于新物距与旧物距之比 $9/4$ 。焦距的增大弥补了物距增加导致的像缩小, 实现了像高的守恒, 这正是 Dolly Zoom 效果的核心机制。
- **`compute_f_pos` 中, 摄像机需要后退以增大两物体的像高差:** 在测试用例中, 初始状态下物体 1 与物体 2 的像高分别为 400 px 和 120 px, 比值约为 3.33; 要将比值调整为 2, 需要让物体 2 的像高相对增大, 因此摄像机后退 ($pos = -4$), 使物体 2 的像高从 120 px 增大到 200 px, 而物体 1 的像高始终维持在 400 px。
- **两物体的像高之比仅由摄像机位置决定, 与焦距无关:** 这一结论来自于推导过程中 f 的自然约去 (见公式 (12)), 是 Dolly Zoom 技巧能够成立的数学基础——改变焦距只能整体缩放图像, 不会改变不同深度物体之间的相对大小; 只有改变摄像机位置, 才能调整它们的相对大小关系。
- **当 $H_1 = \text{ratio} \cdot H_2$ 时方程无解:** 公式 (16) 的分母为 $H_1 - \text{ratio} \cdot H_2$, 若此值为零, 说明两物体在图像中的大小之比恰好等于其物理高度之比, 无论摄像机在哪里, 两者的像高之比都相同, 无法通过改变位置来达到特定比值, 此时需要通过改变焦距整体缩放, 但这并不能改变相对比例, 因此该情形下目标无法实现。

5 总结

本实验通过推导和实现两个函数, 完整地实践了 Dolly Zoom 效果的数学原理:

- **推导了小孔成像模型下的像高公式:** $h = f \cdot H / Z$, 明确了焦距、物距与像高之间的定量关系, 为后续计算提供了理论基础。
- **实现了 `compute_focal_length`:** 在已知摄像机位移的情况下, 利用“像高不变”约束, 解析地求出新焦距 $f = f_{\text{ref}} \cdot (d_{\text{ref}} - pos) / d_{\text{ref}}$ 。该函数支持数组输入, 可高效地为动画每一帧生成对应焦距。
- **实现了 `compute_f_pos`:** 在“物体 1 像高不变”与“两物体像高之比等于目标值”两个约束下, 通过联立方程解耦求解, 先利用比例约束解出 pos , 再代入求出 f 。
- **验证了关键结论:** 两物体在图像中的相对大小只受摄像机位置影响, 与焦距无关; 焦距的作用是整体缩放图像, 而不能改变不同深度物体之间的相对大小。这正是 Dolly Zoom 效果中位置和焦距各自承担独立职责的原因。

通过本实验, 对透视投影变换的理解从抽象的矩阵公式落实到了具体的视觉效果, 认识到小孔成像模型中各参数之间精妙的补偿关系是许多计算机视觉和电影摄影技巧的数学基础。

附录：实验源代码

A.1 compute_f_pos

```
def compute_f_pos(d1_ref, d2_ref, H1, H2, ratio, f_ref):
    """
    compute camera focal length and camera position to achieve certain
        ratio
    between objects

    Input:
    - d1_ref: distance of the first object
    - d2_ref: distance of the second object
    - H1: height of the first object in physical world
    - H2: height of the second object in physical world
    - ratio: ratio between two objects (h1/h2)
    - f_ref: 1 by 1 double, previous camera focal length
    Output:
    - f: 1 by 1, camera focal length
    - pos: 1 by 1, camera position
    """
    # From  $h_1/h_2 = \text{ratio}$ :
    #  $[f \cdot H_1 / (d1\_ref - pos)] / [f \cdot H_2 / (d2\_ref - pos)] = \text{ratio}$ 
    #  $H_1 \cdot (d2\_ref - pos) = \text{ratio} \cdot H_2 \cdot (d1\_ref - pos)$ 
    #  $pos = (H_1 \cdot d2\_ref - \text{ratio} \cdot H_2 \cdot d1\_ref) / (H_1 - \text{ratio} \cdot H_2)$ 
    pos = (H1 * d2_ref - ratio * H2 * d1_ref) / (H1 - ratio * H2)

    # From  $h_1$  constant:  $f\_ref \cdot H_1 / d1\_ref = f \cdot H_1 / (d1\_ref - pos)$ 
    f = f_ref * (d1_ref - pos) / d1_ref

    return f, pos
```

A.2 compute_focal_length

```
def compute_focal_length(d_ref, f_ref, pos):
    """
    compute camera focal length using given camera position

    Input:
    - d_ref: 1 by 1 double, distance of the object whose size remains
```

```
    constant
- f_ref: 1 by 1 double, previous camera focal length
- pos: 1 by n, each element represent camera center position on the z
  axis.
Output:
- f: 1 by n, camera focal length
"""
# Keep image height constant: f_ref*H/d_ref = f*H/(d_ref-pos)
f = f_ref * (d_ref - pos) / d_ref
return f
```

A.3 测试代码

```
# 测试 compute_f_pos
d1_ref = 4
d2_ref = 20
H1 = 4
H2 = 6
ratio = 2
f_ref = 400
f, pos = compute_f_pos(d1_ref, d2_ref, H1, H2, ratio, f_ref)
# 期望输出: f = 800, pos = -4

# 测试 compute_focal_length
d_ref = 4
f_ref = 400
pos = -5
f = compute_focal_length(d_ref, f_ref, pos)
# 期望输出: f = 900
```